

Python-SQL-Project

Objective

The primary goal of this project was to bridge the gap between raw data storage and strategic decision-making. By integrating SQL for data extraction and Python for advanced analytics, I developed a comprehensive pipeline to uncover trends and patterns within complex datasets.

Key Components of My Work

- **Data Orchestration:** Managed large-scale datasets using SQL to perform complex joins, aggregations, and subqueries.
- **Advanced Analytics:** Leveraged Python (Pandas/NumPy) to clean, manipulate, and perform statistical analysis on the extracted data.
- **Visual Intelligence:** Created dynamic Data Visualizations (using Matplotlib/Seaborn) to translate technical findings into intuitive, easy-to-understand charts

Basic Level Questions

Q1. List all unique cities where customers are located.

Python:

```
unique_cities = customers['customer_city'].unique()
print(f"Total Unique Cities: {len(unique_cities)}")
```

Output:

```
Total Unique Cities: 4119
```

SQL:

Code:

```
/*Basic Question*/

/*Que1: List all unique cities where customers are located*/
▶ SELECT COUNT(DISTINCT customer_city) AS total_unique_cities
   FROM customers;
```

Output:

	total_unique_cities
▶	4119

Q2. Count the number of orders placed in 2017.

Python:

Code:

```
orders_2017 = orders[orders['order_purchase_timestamp'].dt.year == 2017].shape[0]
print(f"Orders placed in 2017: {orders_2017}")
```

Output:

```
Orders placed in 2017: 45101
```

SQL:

Code:

```
/*Que2: Count the number of orders placed in 2017*/
SELECT
    YEAR(order_purchase_timestamp) AS order_year,
    COUNT(distinct order_id) AS total_orders
FROM orders
WHERE YEAR(order_purchase_timestamp) = 2017
GROUP BY YEAR(order_purchase_timestamp);
```

Output:

order_year	total_orders
2017	45101

Q3. Find the total sales per category.

Python:

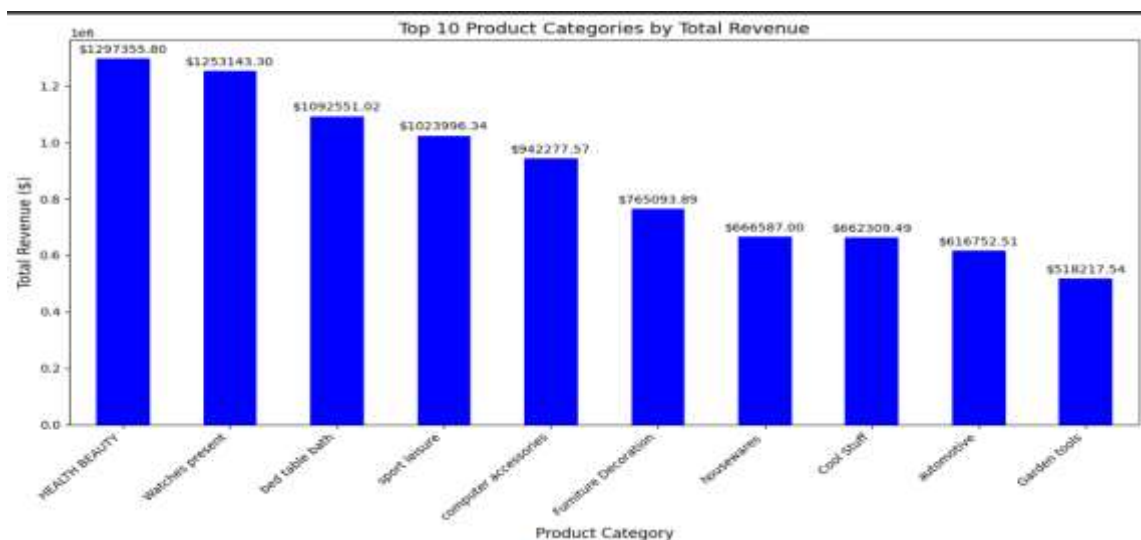
Code:

```
df = pd.merge(products, order_items, on='product_id')
df = pd.merge(df, payments, on='order_id')
top_revenue_cat = df.groupby('product_category')['price'].sum().sort_values(ascending=False).head(10)

plt.figure(figsize=(12, 7))
ax = top_revenue_cat.plot(kind='bar', color='blue')
ax.bar_label(ax.containers[0], fmt='%.2f', padding=3)
plt.title('Top 10 Product Categories by Total Revenue', fontsize=14)
plt.xlabel('Product Category', fontsize=12)
plt.ylabel('Total Revenue ($)', fontsize=12)
plt.xticks(rotation=45, ha='right')

plt.tight_layout()
plt.show()
```

Output:



SQL:

Code:

```
/*Que3: Find the total sales per category*/  
SELECT  
    p.product_category,  
    SUM(oi.price) AS total_revenue  
FROM products p  
JOIN order_items oi ON p.product_id = oi.product_id  
JOIN payments pay ON oi.order_id = pay.order_id  
GROUP BY p.product_category  
ORDER BY total_revenue DESC  
LIMIT 10;
```

Output:

product_category	total_revenue
HEALTH BEAUTY	1297355.79999999633
Watches present	1253143.29999999933
bed table bath	1092551.020000044
sport leisure	1023996.3400000429
computer accessories	942277.5700000358
Furniture Decoration	765093.8900000468
housewares	666587.0000000276
Cool Stuff	662309.4900000015
automotive	616752.5100000096
Garden tools	518217.5400000148

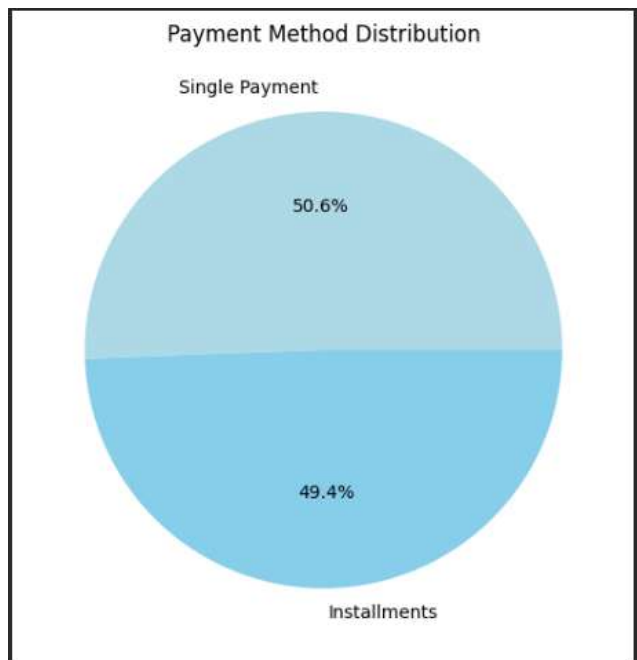
Q4. Calculate the percentage of orders that were paid in more than one installment.

Python:

Code:

```
inst_count = payments['payment_installments'].apply(lambda x: 'Installments' if x > 1 else 'Single Payment').value_counts()  
plt.figure(figsize=(6, 6))  
inst_count.plot(kind='pie', autopct='%1.1f%%', colors=['lightblue', 'skyblue'])  
plt.title('Payment Method Distribution')  
plt.ylabel('')  
plt.show()
```

Output:



SQL:

Code:

```
/*Que4: Calculate the percentage of orders that were paid in installments*/  
SELECT  
    CASE  
        WHEN payment_installments > 1 THEN 'Installments'  
        ELSE 'Single Payment'  
    END AS payment_type,  
    COUNT(*) AS total_count,  
    ROUND(COUNT(*) * 100.0 / SUM(COUNT(*)) OVER(), 1) AS percentage  
FROM payments  
GROUP BY 1;
```

Output:

payment_type	total_count	percentage
Installments	51338	49.4
Single Payment	52548	50.6

5. Count the number of customers from each state.

Python:

Code:

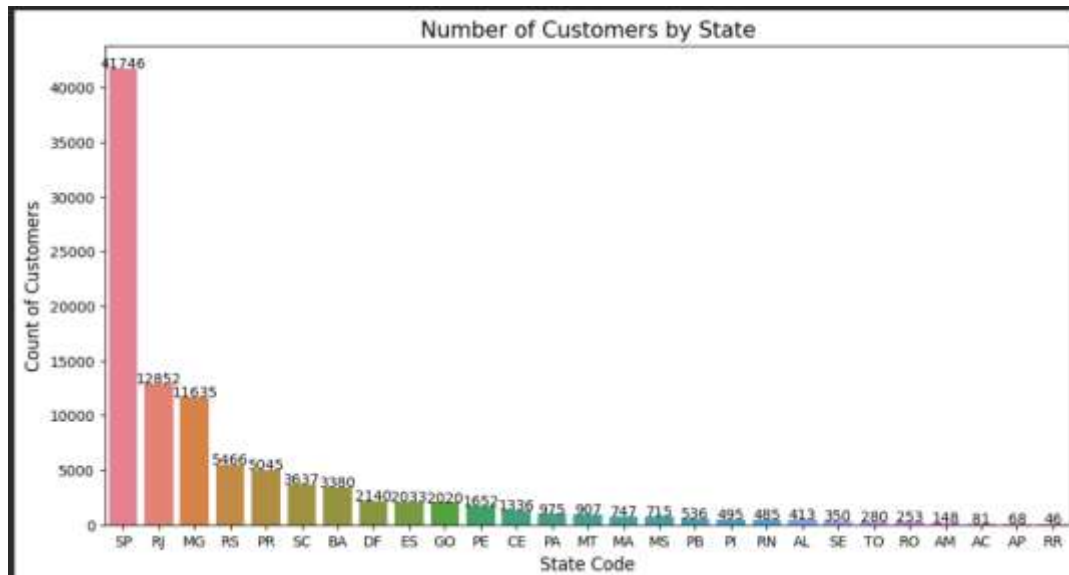
```
plt.figure(figsize=(12, 6))
sns.barplot(x=state_counts.index, y=state_counts.values, hue=state_counts.index, legend=False)

plt.title('Number of Customers by State', fontsize=15)
plt.xlabel('State Code', fontsize=12)
plt.ylabel('Count of Customers', fontsize=12)

for i, value in enumerate(state_counts.values):
    plt.text(i, value + 50, str(value), ha='center', fontsize=10)

plt.show()
```

Output:



SQL:

Code:

```
/*Que5: Count the number of customers from each state*/
SELECT
    customer_state,
    COUNT(customer_id) AS total_customers
FROM customers
GROUP BY customer_state
ORDER BY total_customers DESC
limit 10;
```

Output:

customer_state	total_customers
SP	41746
RJ	12852
MG	11635
RS	5466
PR	5045
SC	3637
BA	3380
DF	2140
ES	2033
GO	2020

Intermediate Level Questions

Q1. Orders per Month in 2018.

Python:

Code:

```
orders_2018 = orders[orders['order_purchase_timestamp'].dt.year == 2018].copy()

orders_2018['month_name'] = orders_2018['order_purchase_timestamp'].dt.strftime('%B')
orders_2018['month_num'] = orders_2018['order_purchase_timestamp'].dt.month

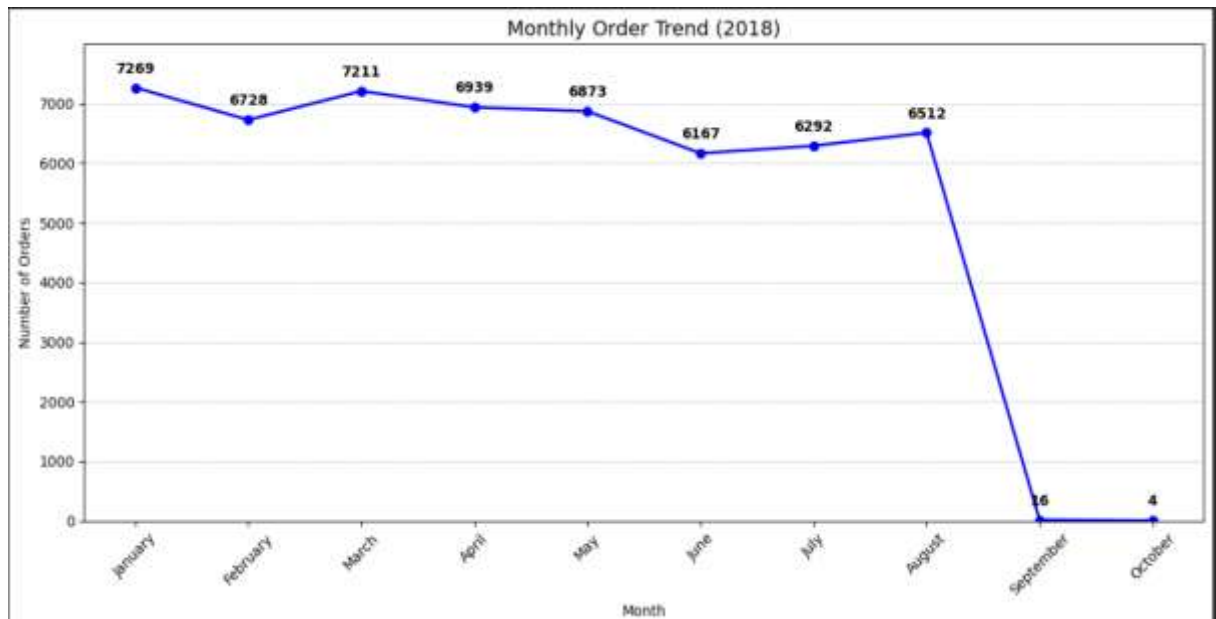
monthly_orders = orders_2018.groupby(['month_num', 'month_name']).size().reset_index(name='count')

plt.figure(figsize=(12, 6))
plt.plot(monthly_orders['month_name'], monthly_orders['count'], marker='o', color='blue', linewidth=2)

for i, txt in enumerate(monthly_orders['count']):
    plt.annotate(f'{txt}', (monthly_orders['month_name'][i], monthly_orders['count'][i]),
                textcoords="offset points", xytext=(0,10), ha='center', fontsize=10, fontweight='bold')

plt.title('Monthly Order Trend (2018)', fontsize=14)
plt.ylabel('Number of Orders')
plt.xlabel('Month')
plt.xticks(rotation=45)
plt.grid(True, axis='y', linestyle='--', alpha=0.7)
plt.ylim(0, monthly_orders['count'].max() * 1.1)
plt.tight_layout()
plt.show()
```

Output:



SQL:

Code:

```
/*Que1: Calculate the number of orders per month in 2018 */  
SELECT  
    MONTHNAME(order_purchase_timestamp) AS month_name,  
    COUNT(distinct order_id) AS total_orders  
FROM orders  
WHERE YEAR(order_purchase_timestamp) = 2018  
GROUP BY month_name  
ORDER BY MIN(MONTH(order_purchase_timestamp));
```

Output:

month_name	total_orders
January	7269
February	6728
March	7211
April	6939
May	6873
June	6167
July	6292
August	6512
September	16
October	4

Q2. Average Number of Products per Order (Grouped by City).

Python:

Code:

```
df_city = pd.merge(customers, orders, on='customer_id')
df_city = pd.merge(df_city, order_items, on='order_id')

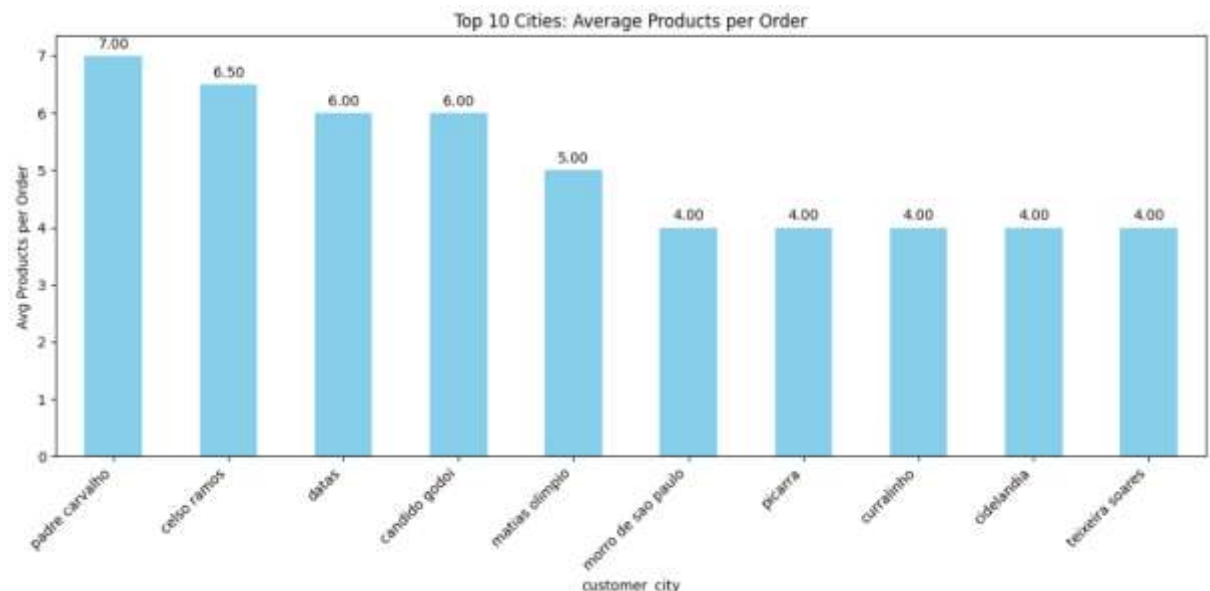
city_stats = df_city.groupby('customer_city').apply(
    lambda x: x['product_id'].count() / x['order_id'].nunique()
).sort_values(ascending=False).head(10)

plt.figure(figsize=(12, 6))
ax = city_stats.plot(kind='bar', color='skyblue')

ax.bar_label(ax.containers[0], fmt='%.2f', padding=3)

plt.title('Top 10 Cities: Average Products per Order')
plt.ylabel('Avg Products per Order')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```

Output:



SQL

Code:

```
/*Que2: Find the average number of products per order, grouped by customer city*/
SELECT
    c.customer_city,
    ROUND(COUNT(oi.product_id) / COUNT(DISTINCT oi.order_id), 2) AS avg_products_per_order
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
JOIN order_items oi ON o.order_id = oi.order_id
GROUP BY c.customer_city
ORDER BY avg_products_per_order DESC
limit 10;
```

Output:

customer_city	avg_products_per_order
padre carvalho	7.00
celso ramos	6.50
candido godoi	6.00
datas	6.00
matias olimpio	5.00
picarra	4.00
cidelandia	4.00
morro de sao paulo	4.00
teixeira soares	4.00
carralinho	4.00

Q3. Percentage of Total Revenue by Product Category.

Python:

Code:

```
rev_per_cat = pd.merge(order_items[['product_id', 'price']],
                       products[['product_id', 'product category']], on='product_id')

total_revenue = rev_per_cat['price'].sum()
category_revenue = rev_per_cat.groupby('product category')['price'].sum()

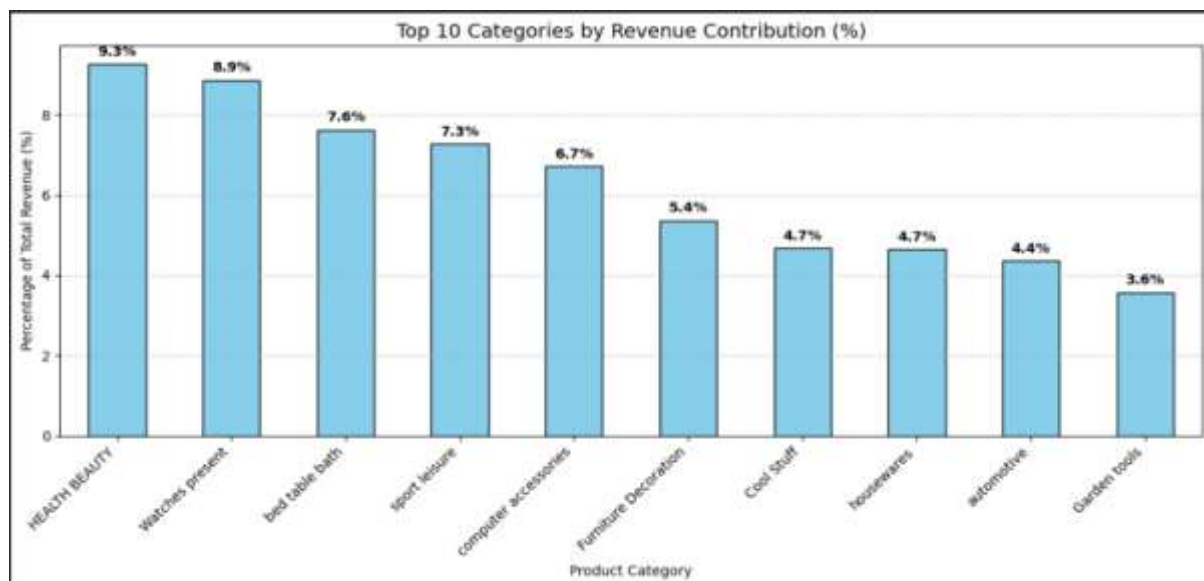
cat_rev_pct = (category_revenue / total_revenue) * 100
cat_rev_pct = cat_rev_pct.sort_values(ascending=False).head(10)

plt.figure(figsize=(12, 6))
ax = cat_rev_pct.plot(kind='bar', color='skyblue', edgecolor='black')

for p in ax.patches:
    ax.annotate(f'{(p.get_height():.1f)}%',
                (p.get_x() + p.get_width() / 2., p.get_height()),
                ha='center', va='center',
                xytext=(0, 9),
                textcoords='offset points',
                fontsize=10, fontweight='bold')

plt.title('Top 10 Categories by Revenue Contribution (%)', fontsize=14)
plt.ylabel('Percentage of Total Revenue (%)')
plt.xlabel('Product Category')
plt.xticks(rotation=45, ha='right')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()
```

Output:



SQL:

Code:

```
/*Que3: Calculate the percentage of total revenue contributed by each product category.*/
SELECT
    p.product_category,
    ROUND(
        SUM(oi.price) * 100.0 / SUM(SUM(oi.price)) OVER(),
        1
    ) AS revenue_percentage
FROM products p
JOIN order_items oi ON p.product_id = oi.product_id
GROUP BY p.product_category
ORDER BY revenue_percentage DESC
LIMIT 10;
```

Output:

product_category	revenue_percentage
HEALTH BEAUTY	9.3
Watches present	8.9
bed table bath	7.6
sport leisure	7.3
computer accessories	6.7
Furniture Decoration	5.4
Cool Stuff	4.7
housewares	4.7
automotive	4.4
toys	3.6

Q4. Correlation Between Product Price and Purchase Count.

Python:

Code:

```
plot_df = product_stats.copy()

plot_df['label'] = plot_df['product_id'].astype(str).str[:5]

fig, ax1 = plt.subplots(figsize=(12, 7))

bars = ax1.bar(plot_df['label'], plot_df['purchase_count'], color='salmon', alpha=0.7)
ax1.set_ylabel('Purchase Count', fontsize=12)

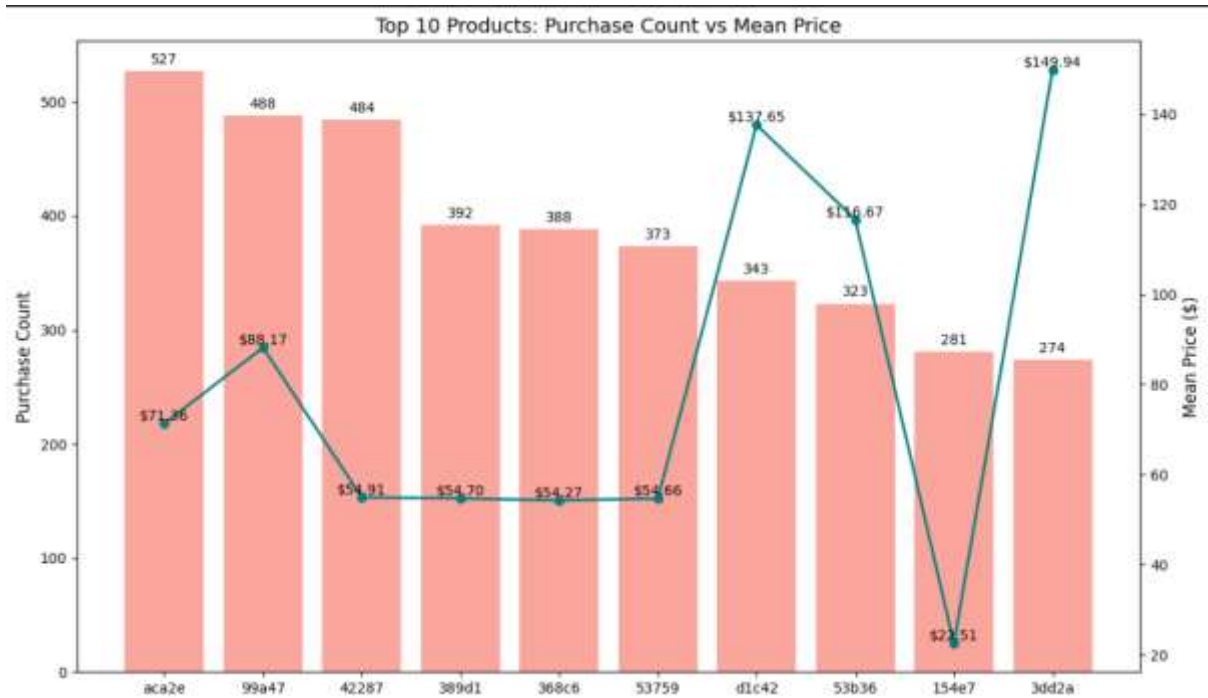
ax1.bar_label(bars, padding=3)

ax2 = ax1.twinx()
ax2.plot(plot_df['label'], plot_df['mean_price'], color='teal', marker='o', linewidth=2)
ax2.set_ylabel('Mean Price ($)', fontsize=12)

for i in range(len(plot_df)):
    ax2.text(
        plot_df['label'].iloc[i],
        plot_df['mean_price'].iloc[i],
        f"${plot_df['mean_price'].iloc[i]:.2f}",
        ha='center',
        va='bottom'
    )

plt.title('Top 10 Products: Purchase Count vs Mean Price', fontsize=14)
plt.tight_layout()
plt.show()
```

Output:



SQL:

Code:

```
/*Que4: Identify the correlation between product price and the number of times a product has been purchased*/
SELECT
    p.product_id,
    p.product_category,
    AVG(oi.price) AS mean_price, -- Matches 'price': 'mean'
    COUNT(oi.order_id) AS purchase_count -- Matches 'order_id': 'count'
FROM products p
JOIN order_items oi ON p.product_id = oi.product_id
GROUP BY p.product_id, p.product_category
ORDER BY purchase_count DESC
limit 10;
```

Output:

product_id	product_category	mean_price	purchase_count
aca2eb7d00ea1a7b8ebd4e68314663af	Furniture Decoration	71.36413662239148	527
99a4788cb24856965c36a24e339b6058	bed table bath	88.16713114754174	488
422879e10f46682990de24d770e7f83d	Garden tools	54.91161157024827	484
389d119b48cf3043d311335e499d9c6b	Garden tools	54.69538265306133	392
368c6c730842d78016ad823897a372db	Garden tools	54.27010309278361	388
53759a2ecddad2bb87a079a1f1519f73	Garden tools	54.65737265415559	373
d1c427060a0f73f6b889a5c7c61f2ac4	computer accessories	137.65163265306117	343
53b36df67ebb7c41585e8d54d6772e08	Watches present	116.66693498452052	323
154e7e31ebfa092203795c972e5804a6	HEALTH BEAUTY	22.509572953736512	281
3dd2a17168ec895c781a9191c1e95ad7	computer accessories	149.93649635036573	274

Q5. Total Revenue Generated by Each Seller (Ranked).

Python:

Code:

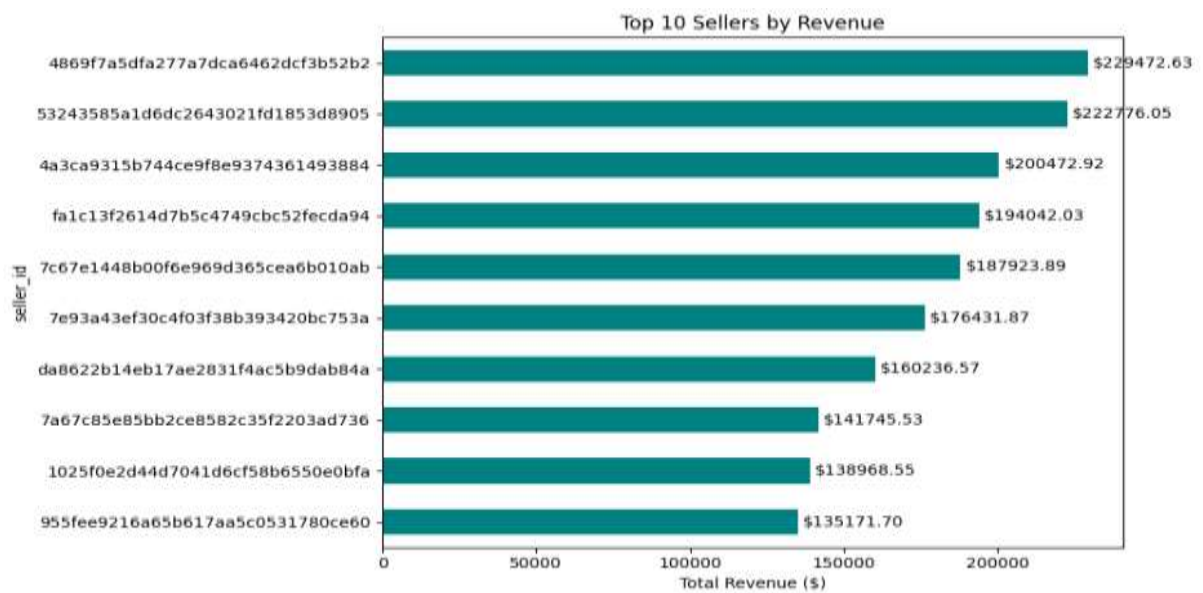
```
seller_rev = order_items.groupby('seller_id')['price'].sum().sort_values(ascending=False).head(10)

plt.figure(figsize=(10, 6))
ax = seller_rev.plot(kind='barh', color='teal')

ax.bar_label(ax.containers[0], fmt='%.2f', padding=3)

plt.title('Top 10 Sellers by Revenue')
plt.xlabel('Total Revenue ($)')
plt.gca().invert_yaxis()
plt.tight_layout()
plt.show()
```

Output:



SQL:

Code:

```
/*Que5: Calculate the total revenue generated by each seller, and rank them by revenue.*/  
SELECT  
    seller_id,  
    SUM(price) AS total_revenue,  
    DENSE_RANK() OVER (ORDER BY SUM(price) DESC) AS seller_rank  
FROM order_items  
GROUP BY seller_id  
limit 10;
```

Output:

seller_id	total_revenue	seller_rank
4869f7a5dfa277a7dca6462dcf3b52b2	229472.62999999981	1
53243585a1d6dc2643021fd1853d8905	222776.04999999952	2
4a3ca9315b744ce9f8e9374361493884	200472.91999999949	3
fa1c13f2614d7b5c4749cbc52fecda94	194042.029999999846	4
7c67e1448b00f6e969d365cea6b010ab	187923.8899999995	5
7e93a43ef30c4f03f38b393420bc753a	176431.86999999982	6
da8622b14eb17ae2831f4ac5b9dab84a	160236.569999999538	7
7a67c85e85bb2ce8582c35f2203ad736	141745.53000000177	8
1025f0e2d44d7041d6cf58b6550ebfa	138968.5499999995	9
955fee9216a65b617aa5c0531780ce60	135171.69999999914	10

Advanced Level Question

Q1. Moving Average of Order Values.

Python:

Code:

```
df = pd.merge(orders, payments, on='order_id')

df['order_purchase_timestamp'] = pd.to_datetime(df['order_purchase_timestamp'])
df = df.sort_values(['customer_id', 'order_purchase_timestamp'])

df['moving_avg'] = (df.groupby('customer_id')['payment_value']
                    .expanding()
                    .mean()
                    .reset_index(level=0, drop=True))

result = df[['customer_id', 'order_purchase_timestamp', 'payment_value', 'moving_avg']]
print(result.head(10))
```

Output:

```
...
customer_id order_purchase_timestamp \
71587 00012a2ce6f8dcda20d059ce98491703 2017-11-14 16:08:26
10466 000161a058600d5901f007fab4c27140 2017-07-16 09:40:32
68795 0001fd6190edaaf884bcdf3d49edf079 2017-02-28 11:06:43
45159 0002414f95344307404f0ace7a26f1d5 2017-08-16 13:09:20
6119 000379cdec625522490c315e70c7a9fb 2018-04-02 13:42:17
76895 0004164d20a9e969af783496f3408652 2017-04-12 08:35:12
48295 000419c5494106c306a97b5635748086 2018-03-02 17:47:40
62657 00046a560d407e99b969756e0b10f282 2017-12-18 11:08:30
82776 00050bf6e01e69d5c0fd612f1bcfb69c 2017-09-17 16:04:44
83947 000598caf2ef4117407665ac33275130 2018-08-11 12:14:35

payment_value moving_avg
71587 114.74 114.74
10466 67.41 67.41
68795 195.42 195.42
45159 179.35 179.35
6119 107.01 107.01
76895 71.80 71.80
48295 49.40 49.40
62657 166.59 166.59
82776 85.23 85.23
83947 1255.71 1255.71
```

SQL:

Code:

```
/*Que1: Calculate the moving average of order values for each customer over their order history*/
SELECT
    customer_id,
    order_purchase_timestamp,
    payment_value,
    AVG(payment_value) OVER (PARTITION BY customer_id ORDER BY order_purchase_timestamp) AS moving_avg
FROM orders o
JOIN payments p ON o.order_id = p.order_id
limit 10;
```

Output:

customer_id	order_purchase_timestamp	payment_value	moving_avg
00012a2ce6f8dcda20d059ce98491703	2017-11-14 16:08:26	114.74	114.74
000161a058600d5901f007fab4c27140	2017-07-16 09:40:32	67.41	67.41
0001fd6190edaaaf884bc3d49edf079	2017-02-28 11:06:43	195.42	195.42
0002414f95344307404f0ace7a26f1d5	2017-08-16 13:09:20	179.35	179.35
000379cdec625522490c315e70c7a9fb	2018-04-02 13:42:17	107.01	107.01
0004164d20a9e969af783496f3408652	2017-04-12 08:35:12	71.8	71.8
000419c5494106c306a97b5635748086	2018-03-02 17:47:40	49.4	49.4
00046a560d407e99b969756e0b10f282	2017-12-18 11:08:30	166.59	166.59
00050bf6e01e69d5c0fd612f1bcfb69c	2017-09-17 16:04:44	85.23	85.23
000598caf2ef4117407665ac33275130	2018-08-11 12:14:35	1255.71	1255.71

Q2. Cumulative Sales per Month for Each Year.

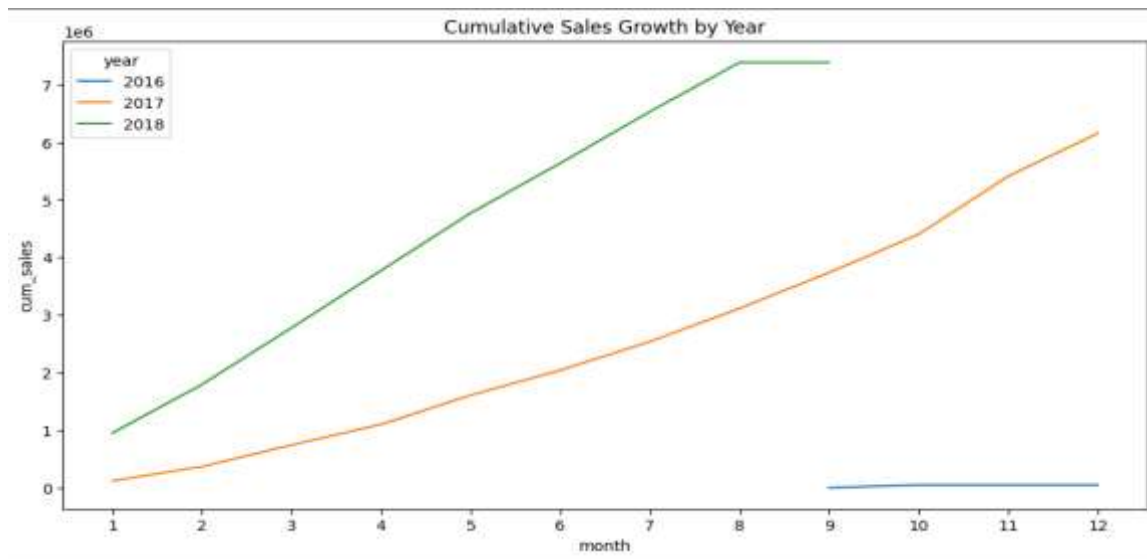
Python:

Code:

```
order_val = order_items.groupby('order_id')['price'].sum().reset_index()
sales_trend = pd.merge(orders[['order_id', 'order_purchase_timestamp']], order_val, on='order_id')
sales_trend['year'] = sales_trend['order_purchase_timestamp'].dt.year
sales_trend['month'] = sales_trend['order_purchase_timestamp'].dt.month
monthly_sales = sales_trend.groupby(['year', 'month'])['price'].sum().reset_index()
monthly_sales['cum_sales'] = monthly_sales.groupby('year')['price'].cumsum()

plt.figure(figsize=(12, 6))
sns.lineplot(data=monthly_sales, x='month', y='cum_sales', hue='year', palette='tab10')
plt.title('Cumulative Sales Growth by Year')
plt.xticks(range(1, 13))
plt.show()
```

Output:



SQL:

Code:

```
SELECT
    YEAR(o.order_purchase_timestamp) AS sales_year,
    MONTH(o.order_purchase_timestamp) AS sales_month,
    SUM(p.payment_value) AS monthly_revenue
FROM orders o
JOIN payments p ON o.order_id = p.order_id
GROUP BY sales_year, sales_month

SELECT
    sales_year, sales_month, monthly_revenue,
    SUM(monthly_revenue) OVER (PARTITION BY sales_year ORDER BY sales_month) AS cumulative_sales
FROM MonthlySales;
```

Output:

sales_year	sales_month	monthly_revenue	cumulative_sales
2017	10	779677.8800000009	5176462.450000014
2017	11	1194882.8000000006	6371345.2500000205
2017	12	878401.4799999993	7249746.730000002
2018	1	1115004.1800000006	1115004.1800000006
2018	2	992463.34000000042	2107467.5200000047
2018	3	1159652.11999999955	3267119.64
2018	4	1160785.47999999953	4427905.1199999995
2018	5	1153982.14999999987	5581887.2699999994
2018	6	1023880.49999999973	6605767.7699999991
2018	7	1066540.74999999988	7672308.519999999
2018	8	1022425.31999999951	8694733.839999985
2018	9	4439.54	8699173.379999984
2018	10	589.6700000000001	8699763.049999984

Q3. Year-over-Year (YoY) Growth Rate.

Python:

Code:

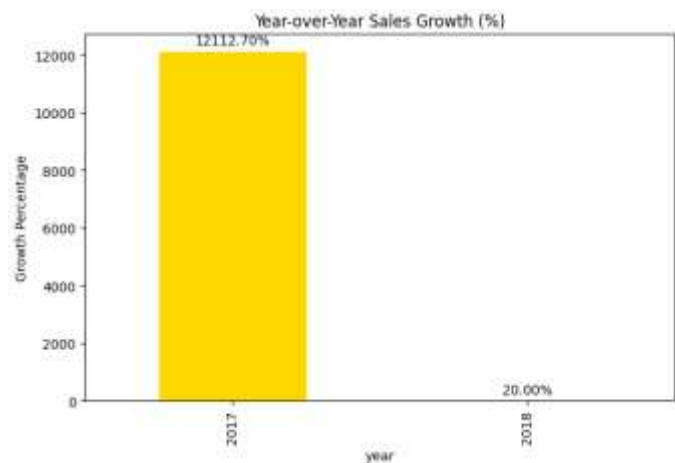
```
df4 = pd.merge(orders, payments, on='order_id')

df4['year'] = pd.to_datetime(df4['order_purchase_timestamp']).dt.year
yearly_sales = df4.groupby('year')['payment_value'].sum()

yoy_growth = yearly_sales.pct_change() * 100

ax = yoy_growth.dropna().plot(kind='bar', figsize=(8, 5), color='gold')
ax.bar_label(ax.containers[0], fmt='%.2f%%', padding=3)
plt.title('Year-over-Year Sales Growth (%)')
plt.ylabel('Growth Percentage')
plt.show()
```

Output:



SQL:

Code:

```
WITH YearlySales AS (
    SELECT
        YEAR(order_purchase_timestamp) AS year,
        SUM(payment_value) AS total_revenue
    FROM orders o
    JOIN payments p ON o.order_id = p.order_id
    GROUP BY year
)
SELECT
    year,
    total_revenue,
    LAG(total_revenue) OVER (ORDER BY year) AS previous_year_revenue,
```

Output:

2016	59362.3399999999975	NULL	NULL
2017	7249746.729999717	59362.3399999999975	12112.703761340474
2018	8699763.049999887	7249746.729999717	20.00092381158571

Q4. Retention Rate (Repeat Purchase within 6 Months).

Python:

Code:

```
first_purchase = orders.groupby('customer_id')['order_purchase_timestamp'].min().reset_index()
first_purchase.columns = ['customer_id', 'first_order_date']

retention_data = pd.merge(orders[['customer_id', 'order_purchase_timestamp']], first_purchase, on='customer_id')

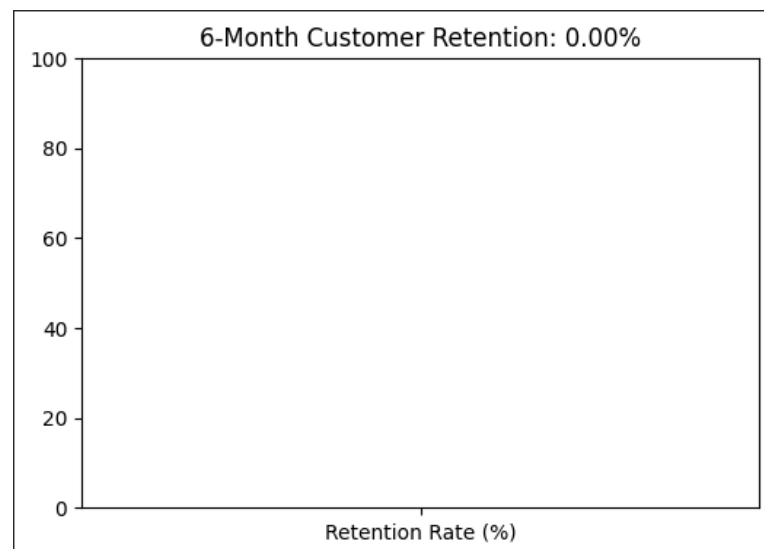
retention_data['days_diff'] = (retention_data['order_purchase_timestamp'] - retention_data['first_order_date']).dt.days

retained_customers = retention_data[(retention_data['days_diff'] > 0) & (retention_data['days_diff'] <= 180)][['customer_id']].nunique()
total_unique_customers = customers['customer_id'].nunique()

retention_rate = (retained_customers / total_unique_customers) * 100

plt.figure(figsize=(5, 4))
plt.bar(['Retention Rate (%)'], [retention_rate], color='seagreen')
plt.ylim(0, 100)
plt.title(f'6-Month Customer Retention: {retention_rate:.2f}%')
plt.show()
```

Output:



SQL:

Code:

```

WITH CustomerStats AS (
    SELECT
        customer_id,
        MIN(order_purchase_timestamp) AS first_order,
        MAX(order_purchase_timestamp) AS last_order,
        COUNT(order_id) AS total_orders
    FROM orders
    GROUP BY customer_id
)
SELECT
    (COUNT(CASE WHEN total_orders > 1
        AND last_order <= DATE_ADD(first_order, INTERVAL 6 MONTH)

```

Output:

retention_rate
0.0000

Q5. Top 3 Customers Who Spent the Most Each Year.

Python:

Code:

```

top_3_per_year['cust_label'] = top_3_per_year['customer_id'].astype(str).str[:8]

plt.figure(figsize=(12, 7))
ax = sns.barplot(data=top_3_per_year, x='year', y='payment_value', hue='cust_label')

for container in ax.containers:
    ax.bar_label(container, fmt='%.0f', padding=3, fontsize=9, rotation=90)

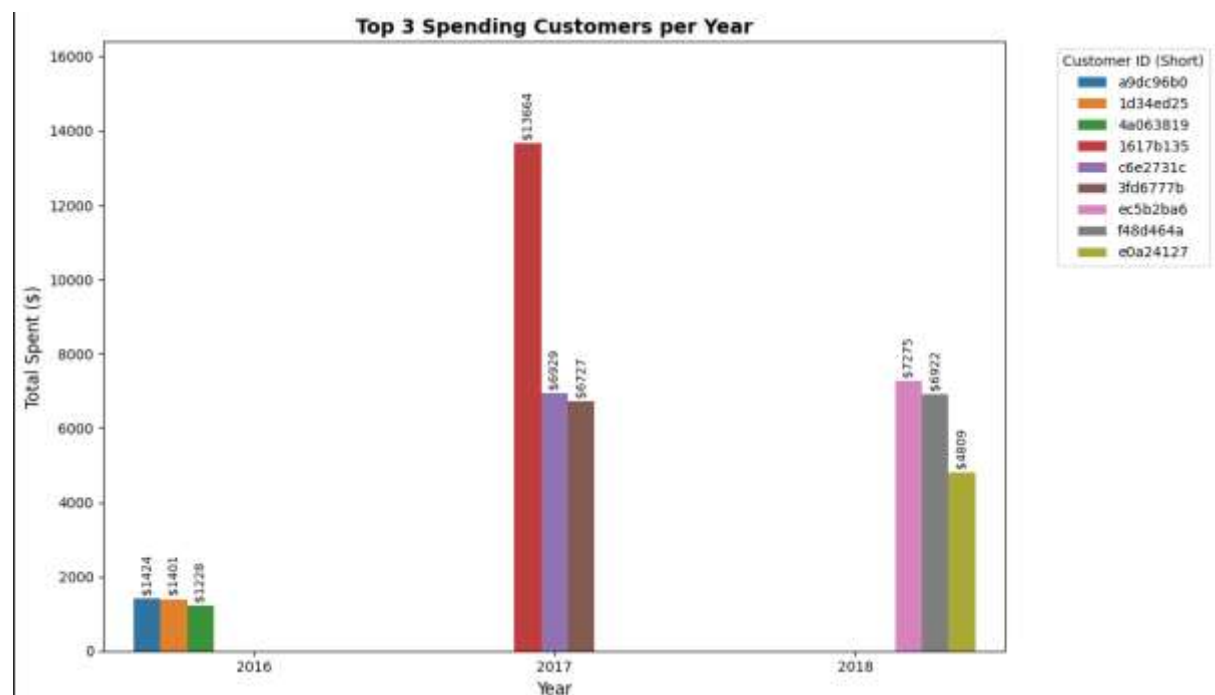
plt.title('Top 3 Spending Customers per Year', fontsize=14, fontweight='bold')
plt.xlabel('Year', fontsize=12)
plt.ylabel('Total Spent ($)', fontsize=12)

plt.margins(y=0.2)

plt.legend(title='Customer ID (Short)', bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()
plt.show()

```

Output:



SQL:

```

/*Que5: Identify the top 3 customers who spent the most money in each year*/
WITH CustomerSpending AS (
    SELECT
        YEAR(o.order_purchase_timestamp) AS year,
        o.customer_id,
        SUM(p.payment_value) AS total_spent
    FROM orders o
    JOIN payments p ON o.order_id = p.order_id
    GROUP BY year, o.customer_id
),
RankedCustomers AS (
    SELECT

```

Output:

2016	a9dc96b027d1252bbac0a9b72d837fc6	1423.55	1
2016	1d34ed25963d5aae4cf3d7f3a4cda173	1400.74	2
2016	4a06381959b6670756de02e07b83815f	1227.78	3
2017	1617b1357756262bfa56ab541c47bc16	13664.08	1
2017	c6e2731c5b391845f6800c97401a43a9	6929.31	2
2017	3fd6777bbce08a352fddd04e4a7cc8f6	6726.66	3
2018	ec5b2ba62e574342386871631fafd3fc	7274.88	1
2018	f48d464a0baaea338cb25f816991ab1f	6922.21	2
2018	e0a2412720e9ea4f26c1ac985f6a7358	4809.44	3